

How to use EEPROM emulation on STM32 MCUs

Introduction

EEPROMs (electrically erasable programmable read-only memories) are used for nonvolatile storage of updatable application data, or to retain small amounts of data in the event of power failure in complex systems. To reduce cost, an external EEPROM can be replaced by on-chip flash memory, provided that a specific software algorithm is used.

This application note describes the software solution (X-CUBE-EEPROM) for substituting a standalone EEPROM by emulating the EEPROM mechanism using the on-chip flash memory available on the STM32 series products listed in [Table 1](#).

X-CUBE-EEPROM also provides a firmware package including examples showing how to exploit this EEPROM emulation driver (see [Section 5: API and application examples](#)).

For STM32WB series products only, an example that maintains a Bluetooth® LE connection and communication while performing EEPROM operations is provided.

For STM32H5 series products, an example using the high-cycle data area for the EEPROM operations is provided.

The emulation method uses at least two flash memory pages, between which the EEPROM emulation code swaps data as they become filled. This is transparent to the user. The EEPROM emulation driver supplied with this application note has the following features:

- Lightweight implementation and reduced footprint.
- A simple API consisting of a few functions to format, initialize, read and write data, and clean up flash memory pages.
- At least two flash memory pages to be used for internal data management.
- Clean-up simplified for the user (background page erase).
- Wear-leveling algorithm to increase emulated EEPROM cycling capability.
- Robust against asynchronous resets and power failures.
- Optional protection implementation for flash memory sharing between cores in multicore STM32 devices (for example STM32WB series).
- Maintenance of cache coherency.

The EEPROM size to be emulated is flexible and only limited by the flash memory size allocated to that purpose.

Table 1. Application products

Type	Series
Microcontrollers	STM32C0 series, STM32G0 series, STM32G4 series, STM32H5 series, STM32L4 series, STM32L4+ series, STM32L5 series, STM32U0 series, STM32U3 series, STM32U5 series, STM32WB series, STM32WL series

1 General information

This document scopes STM32 microcontrollers that are based on an Arm® core.



Note: *Arm is a registered trademark of Arm Limited (or its subsidiaries or affiliates) in the US and/or elsewhere. The Arm word and logo are trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. All rights reserved.*

Reference documents

EEPROM emulation solutions and application notes are available for other STM32 series as listed below.

- [1] Application note STM32F0 series *EEPROM emulation in STM32F0xx microcontrollers* (AN4061)
- [2] Application note STM32F1 series *EEPROM emulation in STM32F10x microcontrollers* (AN2594)
- [3] Application note STM32F2 series *EEPROM emulation in STM32F2xx microcontrollers* (AN3390)
- [4] Application note STM32F3 series *EEPROM emulation in STM32F30x/STM32F31x STM32F37x/STM32F38x microcontrollers* (AN4056)
- [5] Application note STM32F4 series *EEPROM emulation in STM32F40x/STM32F41x microcontrollers* (AN3969)
- [6] Application note *Building wireless applications with STM32WB series microcontrollers* (AN5289)
- [7] Reference manual *STM32H563/H573 and STM32H562 Arm®-based 32-bit MCUs* (RM0481)

2 Main differences between external and emulated EEPROM

EEPROM is a key component of many embedded applications that require nonvolatile storage of data updated with byte, half-word, or word granularity during runtime. However, microcontrollers used in these systems are very often based on embedded flash memory. To eliminate components, save PCB space and reduce system costs, the STM32 flash memory may be used instead of the external EEPROM to store not only code, but also data.

Special software management is required to store data in embedded flash memory. The EEPROM emulation software scheme depends on many factors, including the required EEPROM reliability, the architecture of the flash memory used, and the final product requirements, among other parameters.

The main differences between embedded flash memory and external serial EEPROM are the same for any microcontroller that uses flash memory technology (they are not specific to STM32 series microcontrollers). One difference is that EEPROMs do not require an erase operation to free up space before data can be written again. Other major differences are summarized in [Table 2](#).

Table 2. Differences between external and emulated EEPROM

Feature	External EEPROM (for example, M24C64: I ² C serial access EEPROM)	Emulated EEPROM using on-chip flash memory
Write time	- Random byte write in 4 ms. Word program time = 16 ms - Page (32 bytes) write 4 ms. Consecutive words program time = 500 μs	Word program time: from 90 μ s to 838 ms ⁽¹⁾
Erase time	N/A	2 Kbytes page-erase time: for instance, 22 ms ⁽²⁾
Memory size	From a few Kbytes to 2048 Kbytes	Only limited by the size of flash memory allowed for EEPROM emulation.
Read access	- Serial: 100 μs - Random word: 92 μs - Page: 22.5 μs per byte	Parallel: the access time is from 6 μ s to 592 μ s ⁽¹⁾ .
Endurance	4 million cycles at 25°C 1.2 million cycles at 85°C 600 kilocycles at 125°C	10 kcycles per page at 105°C ⁽²⁾ . Using multiple on-chip flash memory pages is equivalent to increasing the number of write cycles. See Section 4.4: Cycling capability: EEPROM endurance improvement .
Retention ⁽²⁾	50 years at 125°C 100 years at 25°C	7 years at 125°C 15 years at 105°C 30 years at 85°C

1. For further details, refer to [Section 5.3: EEPROM emulation timing](#).

2. Example data for the STM32L4 series. Refer to the datasheet of your STM32 product.

2.1 Difference in write access time

Flash memories have a shorter write access time allowing critical parameters to be stored faster in the emulated EEPROM than in an external EEPROM in most cases. However, due to the data transfer mechanism, the emulated EEPROM write access time sometimes becomes significantly higher.

2.2 Programming and erase operations

Unlike flash memories, EEPROMs do not require an erase operation to free up space before writing to a programmed address. This is a major difference between a standalone EEPROM and an emulated EEPROM using embedded flash memory.

Emulated EEPROM using embedded flash memory

The erase process management is fully handled by the EEPROM emulation software, but the erase operation is left to application software management. This allows a reduction of the worst case write time, and also flash page erase operations when the application execution time becomes less critical.

Moreover, as the flash memory programming and erase operations are quite long, power failures and other spurious events that might interrupt the erase process (such as resets) have been considered when designing the flash memory management software. The EEPROM emulation software has been designed to be robust against power failures and fully asynchronous resets.

Standalone external EEPROM

Once started by the CPU, the writing of a word cannot be interrupted by a CPU reset. Only a supply failure may interrupt the write process, so power supply monitoring and properly sized decoupling capacitors are necessary to secure the complete writing process inside a standalone EEPROM.

3 Implementing EEPROM emulation

3.1 Principle

EEPROM emulation can be performed in various ways, considering the flash memory characteristics and final product requirements. The approach detailed below requires two sets of flash memory pages allocated to nonvolatile data.

The first set of pages is initially erased and used to store new data and flash memory programming operations are done sequentially in increasing order of flash memory addresses. Once the first set of pages is full of data, it needs to be garbage-collected.

The second set of pages collects only the valid data from the first set of pages and the remaining area can be used to store new data. Once the transfer of valid data to the second set of pages is completed, the first set of pages can be erased.

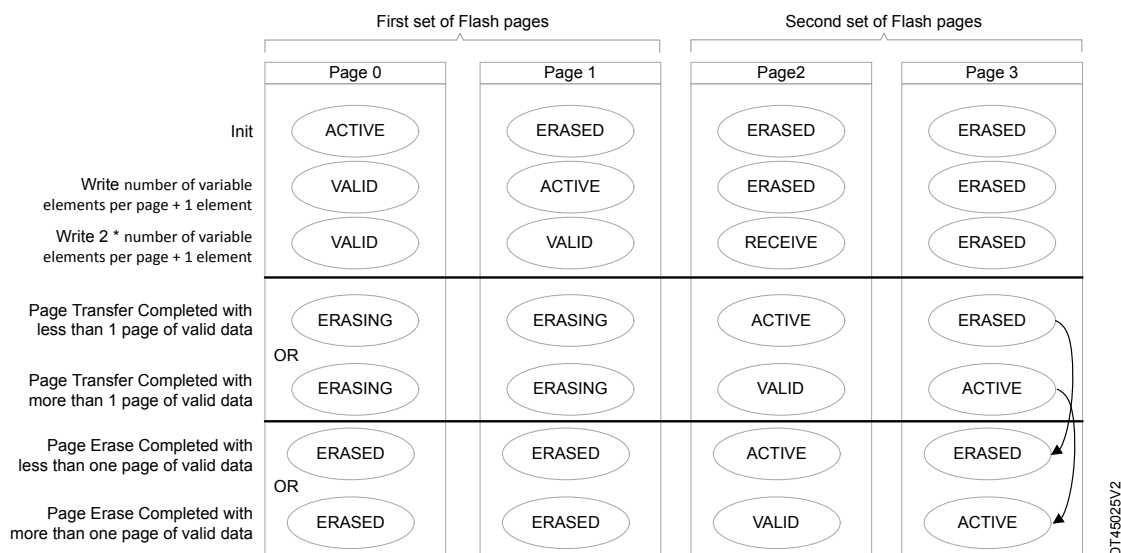
Each set of pages can be made up of one or several flash memory pages. For most STM32 series covered by this document, a header field that occupies the first four 64-bit words (32 bytes) of each page indicates its status, the exception being the STM32U5 series, where the header occupies the first four 128-bit words (64 bytes).

Each page has five possible states:

- **ERASED**: the page is empty (initial state).
- **RECEIVE**: the page used during data transfer to receive data from other full pages.
- **ACTIVE**: the page is used to store new data.
- **VALID**: the page is full. This state does not change until all valid data is completely transferred to the receiving page.
- **ERASING**: valid data in this page has been transferred. The page is ready to be erased.

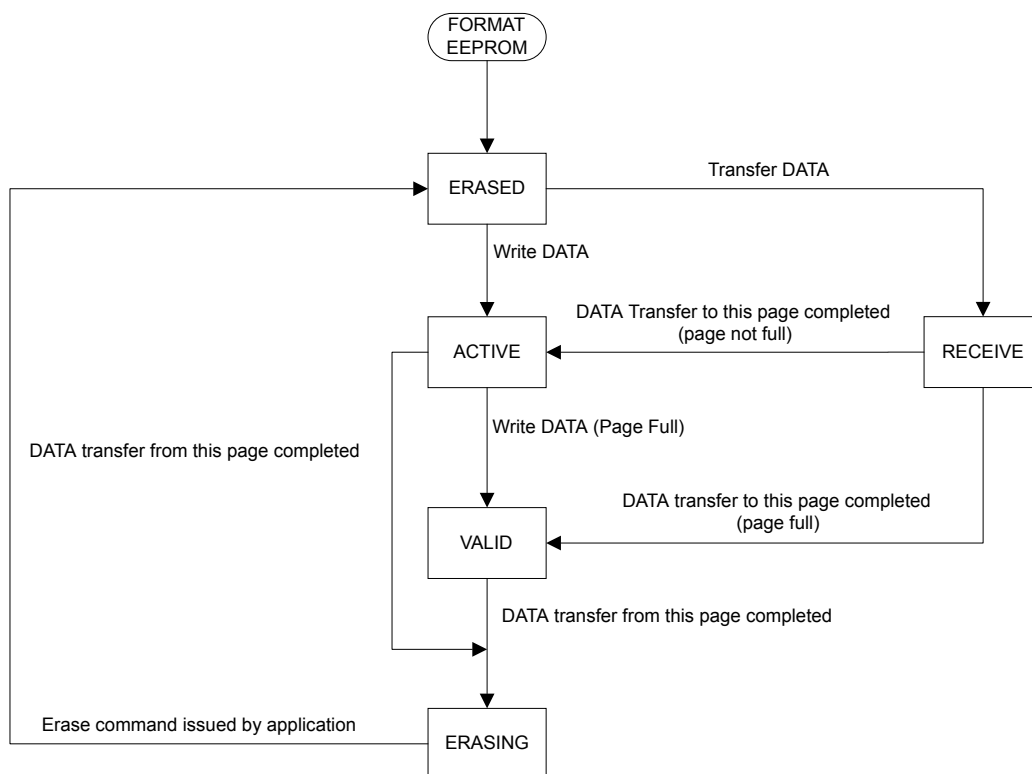
Figure 1 shows the page status evolution in the case where each set of pages is made of two pages.

Figure 1. Page status evolution



3.2 Page status valid transitions

The information provided in this paragraph is only useful for users intending to modify the EEPROM emulation driver. It is not useful for a simple utilization of the driver.

Figure 2. Page status valid transitions


DT45026V1

3.3 Page and variable format

Depending on the STM32 series, flash memory page sizes and composition can vary.

Moreover, for some STM32 series, the EEPROM emulation driver supports either single bank mode, dual bank mode, or both. The right mode must be selected in the EEPROM emulation driver, according to the mode supported. See [Table 3](#) for this information.

Table 3. Flash memory properties

STM32 series/value lines	Flash memory page sizes	Single bank mode support	Dual bank mode support
STM32L4+ series	4 Kbytes (512 words of 64 bits for DBANK = 1)	No ⁽¹⁾	Yes
STM32L41x/42x/43x 44x/45x/46x	2 Kbytes (256 words of 64 bits)	Yes	No
STM32L47x/48x/49x/4Ax	2 Kbytes (256 words of 64 bits)	Yes	Yes
STM32L552/STM32L562	2 Kbytes (256 words of 64 bits) (for DBANK = 1)	No ⁽¹⁾	Yes
STM32G0x0xx and STM32G0x1xx STM32G0B1xx and STM32G0C1xx	2 Kbytes (256 words of 64 bits)	Yes	No Yes
STM32G4 series	2 Kbytes (256 words of 64 bits) (for DBANK = 1)	No ⁽¹⁾	Yes
STM32WB55xx, STM32WB35xx, STM32WB50CG, STM32WB30CE	4 Kbytes (512 words of 64 bits)	Yes	No
STM32WL series	2 Kbytes (8 rows of 256 bytes)	Yes	No
STM32WL3x	2 Kbytes (512 words of 32 bits)	Yes	No
STM32WB15CC, STM32WB10CC	2 Kbytes (256 words of 64 bits)	Yes	No

STM32 series/value lines	Flash memory page sizes	Single bank mode support	Dual bank mode support
STM32U0 series	2 Kbytes (256 words of 64 bits)	Yes	No
STM32U3 series	4 Kbytes (512 words of 64 bits)	Yes	Yes
STM32U5 series	8 Kbytes (512 words of 128 bits)	Yes	Yes
STM32C0x1	2 Kbytes (256 words of 64 bits)	Yes	No
STM32H5 series	8 Kbytes (512 words of 128 bits)	Yes	Yes
STM32H5 series using high-cycle data area ⁽²⁾	6 Kbytes (3072 words of 16 bits)	Yes	Yes

1. When in single bank mode, these devices operate 128 bits wide read accesses. However, the EEPROM emulation solution is designed for 64 bits wide read accesses.
2. The high-cycle data area is available on the STM32H563/H573 and the STM32H562 MCUs.

The minimal write width in flash memory is 64 bits (128 bits for STM32U5 and STM32H5 series, or 16 bits for STM32H5 series using high-cycle data area) due to its ECC (error correcting code) that cannot be switched off. Only zeros (0x0000 0000 0000 0000, or 0x0000 0000 0000 0000 0000 0000 0000 0000 for STM32U5 series) can be written to an already programmed nonnull flash memory line. As the first four words are used by the header, a flash memory page can store up to 252 variable elements when the page size is 2 Kbytes, and up to 508 variable elements when the page size is 4 Kbytes.

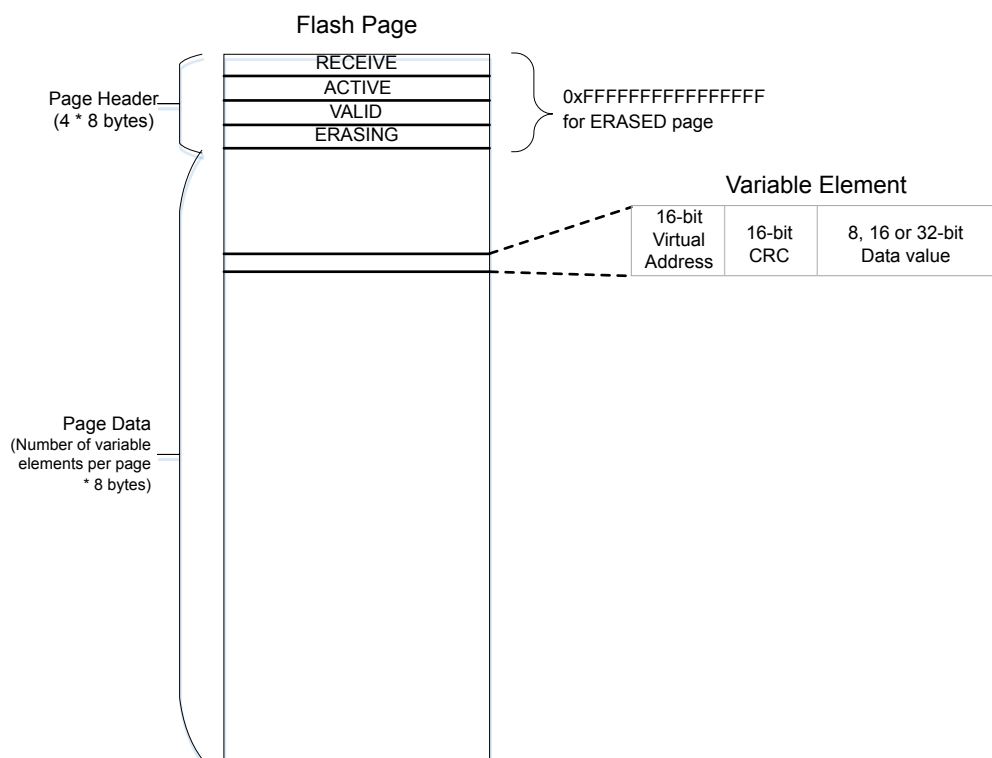
The possible states of a flash memory page are coded by writing 0xAAAA AAAA AAAA AAAA (or 0xAAAA AAAA AAAA AAAA AAAA AAAA AAAA AAAA for the STM32U5 series) into the page header. It is possible to determine the page state using the following procedure:

- The page is in the ERASING state if its fourth line is not erased.
- The page is in the VALID state if the third line is not erased and the fourth line is erased.
- The page is in the ACTIVE state if the second line is not erased and the third and fourth lines are erased.
- The page is in the RECEIVE state if the first line is not erased and the second, third, and fourth lines are erased.
- The page is in the ERASED state if the first four lines are erased.

This algorithm allows the coding of all states and transitions described in [Section 3.2: Page status valid transitions](#). Each variable element is defined by a virtual address and a data value to be stored in flash memory for subsequent retrieval or update. In the implemented software, the virtual address is 16 bits long and the data value is either 8 bits, 16 bits, or 32 bits long. The driver requires the virtual address values to be between 0x0001 (0x0000 corresponds to an EEPROM element invalidated by the driver), and the maximum number of EEPROM variables required. Also, since virtual addresses are 16-bits wide, the maximum number of EEPROM variables cannot exceed 0xFFFF (0xFFFF corresponds to an erased flash memory line). Moreover, the number of variables is limited by the size of the product flash memory (see [Section 4.6: Computing the required size of flash memory for EEPROM emulation](#)).

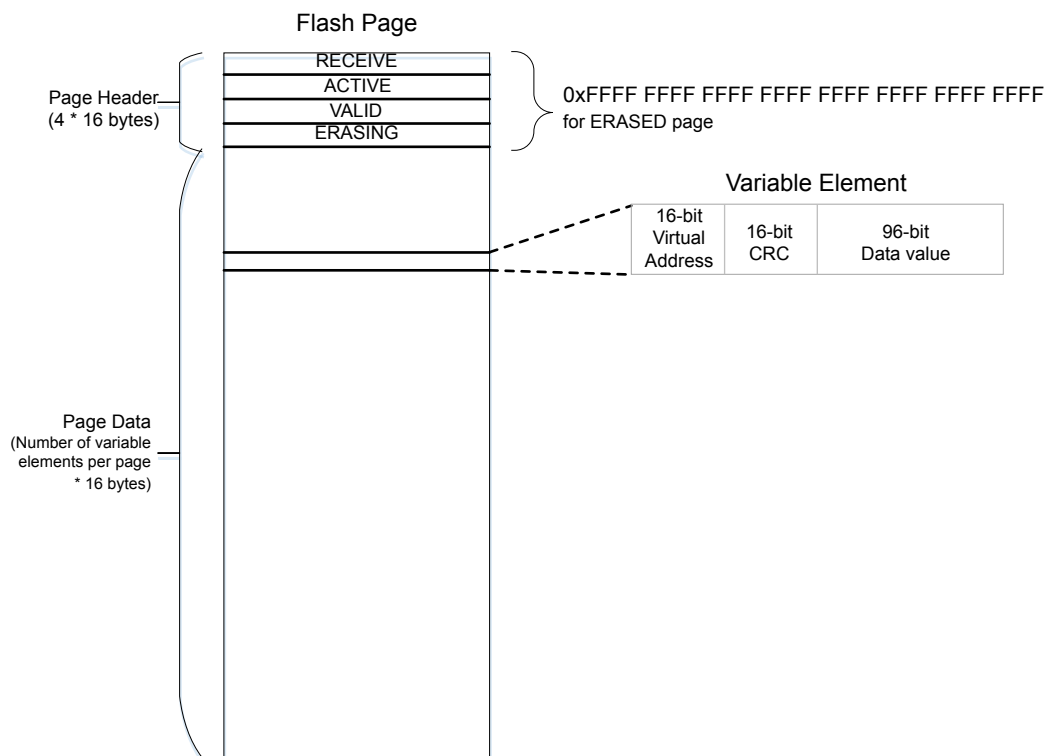
Each element also contains a 16-bit CRC that is used to check the element integrity. When data is modified, the modified data associated with the same virtual address is stored in a new flash memory location. Data retrieval returns the up-to-date data value.

Figure 3. Flash page and EEPROM variable format (except STM32U5 series)



DT4502V2

Figure 4. STM32U5 series Flash page and EEPROM variable format



DT47646V1

3.4 Simple use case

The following example shows the software management for three EEPROM variables with the virtual addresses shown in Table 4.

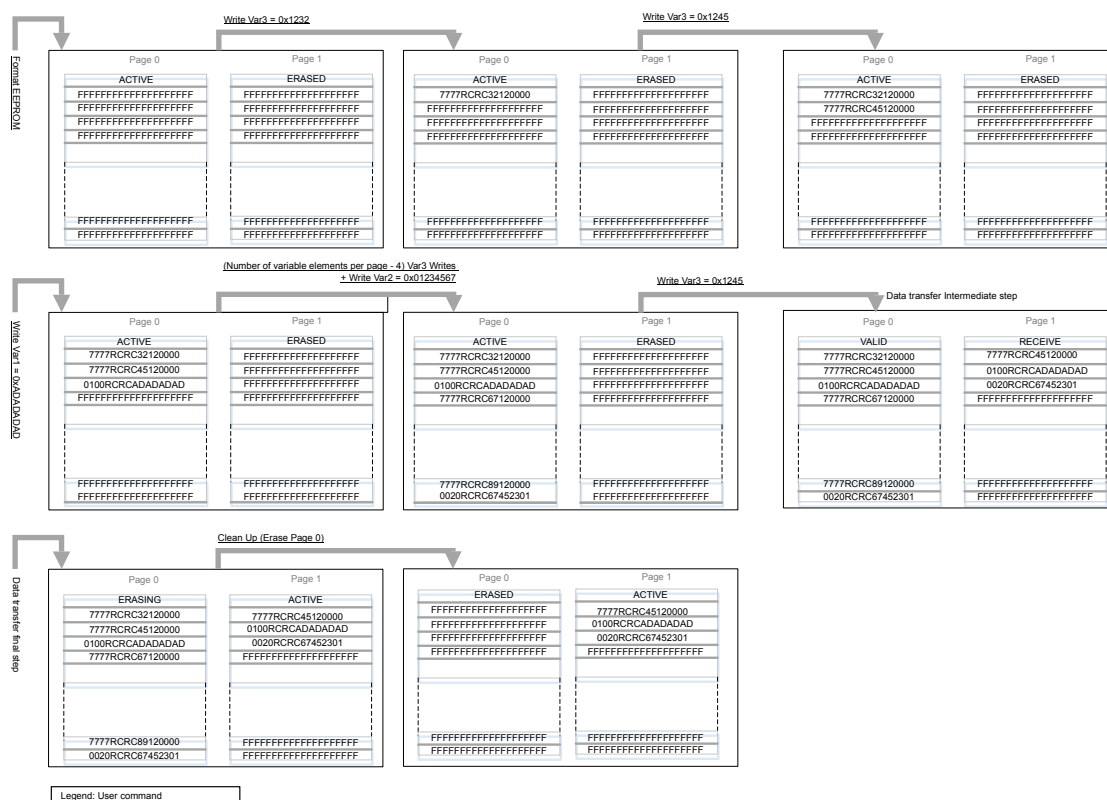
Table 4. Emulated EEPROM virtual addresses

Variable	Virtual address	Data width
Var1	0x0001	32-bit
Var2	0x2000	32-bit
Var3	0x7777	16-bit

For this example, only two flash memory pages are necessary: Page 0 and Page 1. In Figure 5, RCRC represents the 16-bit CRC value for this variable element (refer to Section 4.7: EEPROM emulation robustness for more details about CRC).

Figure 5 shows only the write commands, which are done sequentially in increasing order of flash memory addresses.

Figure 5. Data update flow



3.5 Reading data

Read commands perform flash memory reads from the highest to the lowest address in the ACTIVE or VALID page, and return only valid data. Data is considered valid if it is the latest to be written at a given virtual address, and the integrity check using CRC passes.

Note: Only valid data is copied during the data transfer mechanism.

4 Advanced features

The EEPROM emulation firmware is designed to fulfill most of the requirements for an embedded application in terms of nonvolatile storage. This section covers these requirements in detail. Other embedded application requirements may be needed in particular cases and are addressed in [Section 6: Embedded application aspects](#).

4.1 Data granularity management

Emulated EEPROM can be used in embedded applications where nonvolatile storage of data updated with a byte, half-word, or word granularity is required. Data size generally depends on application requirements such as sensor or communication-interface data size.

The EEPROM emulation firmware is designed to support byte, 16-bit half-word, and 32-bit word granularity (96-bit word granularity for STM32U5 series). However, to optimize the flash memory usage, the user application can gather all smaller sized data elements into 32-bit data elements before storing the content in emulated EEPROM. This would ensure an optimal use of the 64-bit flash memory line by simultaneously writing the 16-bit virtual address, the 16-bit CRC, and the 32-bit data value.

Conversely, it is possible to write the 32-bit data value on two 16-bit half-word with the 16-bit virtual address and the 16-bit CRC, allowing a 32-bit data granularity in the high-cycle data area available in STM32H5 series, which only supports 16-bit half-word granularity by line.

For STM32U5 and STM32H5 series, the user can gather all smaller sized data elements into 96-bit data elements before storing the content in emulated EEPROM. This ensures optimal use of the 128-bit flash memory line by simultaneously writing the 16-bit virtual address, the 16-bit CRC, and the 96-bit data value.

Note: The minimal write width in flash memory is 64 bits (128 bits for STM32U5 and STM32H5 series, or 16 bits for STM32H5 series using high-cycle data area) due to its ECC (error correcting code), commands, which cannot be switched off.

4.2 Wear leveling algorithm and flash memory page allocation

A wear leveling algorithm allows monitoring and even distribution of flash memory write and erase operations between flash memory pages. When no wear-leveling algorithm is used, the pages are not used at the same rate. For instance, pages with long-lived data do not endure as many write and erase cycles as pages that contain frequently updated data. The wear-leveling algorithm ensures that equal use is made of all the available write cycles for each flash memory page.

By design, the EEPROM emulation algorithm distributes evenly the flash memory write and erase operations between flash memory pages. Flash memory writes are performed sequentially in increasing-address order whatever user variable is written. When one set of pages is full, the valid elements are copied to the other set of pages, and the first set of pages is fully erased.

4.3 Guard pages

To even further reduce wear on flash memory pages, the user can decide to add an even number of flash memory guard pages (one guard page per set of pages by default). No guard page is necessary if the number of emulated variables leaves significant room in the ACTIVE page. Increasing this number (4,6,...) increases the flash memory endurance beyond the enabled value. This feature is closely linked to the emulated EEPROM cycling capability - refer to paragraph [Section 4.4: Cycling capability: EEPROM endurance improvement](#).

Taking an example based on the STM32L4, an emulation of 1000 EEPROM variables can be stored in two sets of four flash memory pages (each page being able to store 252 elements). When all elements are written once (or after a page transfer), only eight more elements can be written before a new page transfer is triggered. In this case, the addition of two guard pages is recommended (one per set of pages) so that 260 writes can be performed before a new page transfer is triggered.

4.4 Cycling capability: EEPROM endurance improvement

When EEPROM technology is used, each byte can be individually programmed a finite number of times (typically in the range of 1 million). When flash memory technology is used, it is not possible to write to a nonerased address, and the minimum erase size is the flash memory page size. Consequently, we need to define a program/erase cycle that consists of several flash memory line write accesses followed by one flash memory page erase operation.

Each STM32 device on-chip flash memory page can be programmed and erased reliably a limited number of times. For write-intensive applications that need to update each variable more than this number, the wear leveling algorithm allows the endurance of the emulated EEPROM to be increased.

Table 5 shows the limit number of reliable programming and erasing operations for devices covered by this application note:

Table 5. Flash memory endurance

STM32 series	Flash memory endurance
STM32L4 series	10 kcycles
STM32L4+ series	10 kcycles
STM32L5 series	10 kcycles
STM32G0 series	10 kcycles (1 kcycles for STM32G030x6/x8 and STM32G070CB/KB/RB)
STM32G4 series	10 kcycles
STM32WB series	10 kcycles
STM32WL series	10 kcycles
STM32U0 series	10 kcycles
STM32U3 series	10 kcycles
STM32U5 series	10 kcycles endurance on all flash memories 100 kcycles on up to 256 Kbytes per bank
STM32C0 series	10 kcycles
STM32H5 series	10 kcycles on all flash memories 100 kcycles in high-cycle data area

Knowing the requested size of emulated EEPROM and the targeted endurance, it is possible to compute the flash memory size to be used for that purpose. The flash memory size is also a function of the data width of stored variables.

Table 6. Flash usage for a 4000-byte emulated EEPROM (STM32L4/L4+)

Data width	Number of pages needed for 10 kcycles endurance	Flash size for 10 kcycles endurance	Number of pages needed for 100 kcycles endurance	Flash size for 100 kcycles endurance
STM32L4				
8-bit	34	68 Kbytes	322	644 ⁽¹⁾ Kbytes
16-bit	18	36 Kbytes	162	324 Kbytes
32-bit	10	20 Kbytes	82	164 Kbytes
STM32L4+				
8-bit	18	72 Kbytes	162	648 Kbytes
16-bit	10	40 Kbytes	82	328 Kbytes
32-bit	6	24 Kbytes	42	168 Kbytes

1. Not applicable for STM32L4: a maximum of 512 Kbytes can be allocated to EEPROM emulation. Refer to the STM32 product datasheet for the size of each bank.

Note:

To get the same information for other STM32 series, you can apply the formula presented in Section 4.6: *Computing the required size of flash memory for EEPROM emulation*.

4.5 High-cycle data area of flash memory

The embedded flash memory of STM32H563/H573 and STM32H562 MCUs offers up to 96 Kbytes (maximum) memory area with high cycling capability (100 kcycles) to store data and emulate EEPROM. It can be accessed through the AHB system port from the following address: 0x0900_0000 to 0x0901_7FFF. It is mapped in the eight (or four) last sectors of Bank1 and 2. This area is protected by a robust 6-bit ECC, which enables a read and write granularity of 16 bits at the expense of having sector size shrunk to 6 Kbytes.

A threshold per bank (EDATA(1/2)_STRT) is programmable in user option bytes to determine the beginning of the data flash area. By default, all flash memories are used for code. For example, if 48 Kbytes of data are needed in Bank1, set EDATA1_EN to 1, and EDATA1_STRT to 7. If no data are needed in Bank2, set EDATA2_EN to 0. In this case, the data are accessible from address 0x0900_0000 to 0x0900_BFFF for Bank1. For greater efficiency, it is always recommended to use sector on the other bank for flash memory high-cycle data, so that the application benefits from the RWW capability of the dual-bank arrangement. When the SWAP_BANK feature is enabled, the banks are swapped. The flash memory high-cycle data in Bank 2 are accessible from 0x0900_000 to 0x0900_BFFF, and the data in Bank 1 are accessible from 0x0900_C000 to 0x0901_7FFF.

A bus error is generated on:

- Attempt to access an address between 0x0900_0000 to 0x0901_7FFF while this address is not valid (EDATA(1/2)_EN not enabled or EDATA(1/2)_STRT not correct).
- Attempt to fetch instructions from flash memory high-cycle data area. Erasing the data area sector is possible by normal erase request for the corresponding user flash memory sector (120-127).

Note: More information about the high-cycle data area and its protections are documented in [7]

4.6 Computing the required size of flash memory for EEPROM emulation

As first approximation, the required size of flash memory is proportional to the emulated EEPROM size and to the cycling capability. Table 6 can be used to estimate the required size of flash memory according to the data width of stored elements for STM32L4 series and STM32L4+ series MCUs.

For a precise flash memory size requirement, whatever the STM32 series, use the formula below:

$$\text{Flash size} = \left\lceil \frac{\text{Number of EEPROM variables}}{\text{Variable elements per page}} \times \frac{\text{Number of cycles}}{\text{Flash memory endurance (kcycles)}} \times 2 + \text{Number of guard pages} \right\rceil \times \text{Page size}$$

Where $\lceil \quad \rceil$ represents the next higher integer

Note: Values of 'flash memory endurance values (kcycles)' for applicable STM32 series are listed in Table 5.

The formula is decomposed into the following steps:

1. Knowing the number of elements a flash memory page can store (refer to Table 3) compute the number of flash memory pages needed to store the EEPROM elements. The calculated flash memory size is independent of the element data width.
2. Multiply the number of pages by the targeted integer cycling ratio, for instance 1 for the standard 10 kcycles endurance or 10 to achieve 100 kcycles endurance when the flash memory endurance is 10 kcycles. The flash memory endurance of the product indicated in Table 5 must be considered.
3. Multiply the number of flash memory pages by 2 to include the data transfer mechanism.
4. Add the number of guard pages (an even number) to get the final number of flash memory pages.
5. Knowing the flash memory page size for your product (refer to Table 3), compute the flash memory size required by the EEPROM emulation.

STM32L4 calculation example

To store 4000 individual bytes, and knowing that each page can store up to 252 elements, a set of pages must comprise 16 flash memory pages. A second set of pages, of the same size, is required to transfer data when the first one is full. If we assume that we use two guard pages, 34 flash memory pages are necessary.

Note: This calculation is a slightly conservative estimation.

4.7 EEPROM emulation robustness

In an embedded application, it is possible that a power failure or asynchronous reset might occur while programming or erasing the flash memory. In this case, the content of the flash memory line (if programming) or the complete flash memory page (if erasing) is unknown. There can be a single- or a multiple-bit error, with or without error correcting code detection (ECCD) or correction (ECCC).

Note: The hardware ECC is designed to detect and correct errors after flash memory wear, but not to detect flash memory program or erase operation interruptions reliably. ECC is not available on STM32C0 series.

The EEPROM emulation software is designed and validated to be robust against power failures and asynchronous resets. This robustness relies on several features in the flash memory interface and EEPROM emulation driver that are detailed in the following paragraphs. Refer to the product errata sheet for the flash memory limitations and workarounds.

4.7.1 Data recovery

To detect corrupted data (virtual address and/or data value), a 16-bit CRC (cyclic redundancy check) has been implemented. It is based on the ANSI CRC-16 with the following polynomial: $x^{15} + x^2 + 1$ (represented as 0x8005). Even though this polynomial offers a very high detection rate, it can be modified by the user in the EEPROM configuration file.

When writing a variable element, it is stored with its corresponding CRC value. When reading or transferring a variable element, the CRC is computed and checked against the value stored in the variable element. If it matches, the variable element is considered as valid. In the case of a mismatch, the variable element is invalidated.

Note: The CRC peripheral is used by the EEPROM emulation software to accelerate the CRC computation.

The data recovery feature relies on the fact that all-zeros, and only a full 64-bit zero (128 for STM32U5 series), can be written to an already programmed nonnull word. Invalidating a variable element is done by writing zeros in the variable element. As a consequence, this precludes the use of 0x0000 as a virtual address for the variable element.

Note: The virtual address 0xFFFF is also forbidden as it corresponds to an erased flash memory line.

The data recovery mechanism also requires the user to write zeros in case an ECCD NMI is triggered on this flash memory line. This is achieved by calling the `FI_DeleteCorruptedFlashAddress` function from the NMI service routine in case the ECC detects an error that it cannot correct (that is, the ECCD bit is set).

This mechanism is only applied during the cleanup phase operated in the `EE_Init` function. After this phase, it is assumed that there are no further NMI trigger events.

It is possible that, for some reason, the zero programming operation fails and that a flash memory programming error flag (PROGERR, PGAERR, or PGSERR) is raised (for example if the flash memory line was already at 0, PROGERR is raised). The line then is considered invalid and is kept in its current state (the NMI service routine exits without doing anything).

If flags other than PROGERR, PGAERR or PGSERR are set, the NMI routine enters an infinite loop.

If, despite the cleanup phase, another NMI is triggered during the EEPROM emulation, nothing is done. The fact that the NMI is raised is considered to invalidate the line. The cleanup phase eliminates the majority of lines corrupted by an asynchronous reset or power down. Thus, after the cleanup phase, the number of remaining NMIs may, in most cases, be 0.

If during the first write to a variable element with a given address, the write is interrupted by a power failure or an asynchronous reset, the driver considers that there is no data at this address. In all other cases, the EEPROM emulation software always returns the latest valid data value by finding the previous value for this data stored in flash memory.

4.7.2 Page header recovery

Similarly to data corruption, the page header can be corrupted in the case of power loss or asynchronous reset during a header update or a flash memory page erase.

To detect this corruption and recover from it, the `EE_Init` routine is implemented. It is called immediately after power-up. This routine checks the sequence of page statuses for integrity and performs a repair if necessary. This ensures that no data is lost. Refer to [Section 5.3: EEPROM emulation timing](#) for more details.

To avoid a possible failure scenario, erased pages are systematically erased again upon reset (in the `EE_Init` routine). This ensures a safe behavior but consumes cycling capability of the emulated EEPROM. This systematic erase can be avoided if the application is designed not to generate an asynchronous reset or power failure during flash memory writes or erases. Refer to [Section 6.3: Detecting power failures](#) for more details.

Robustness is achieved at the expense of several checks and recovery mechanisms implemented in the `EE_Init` routine. This induces a slight code size increase and longer initialization time when compared with simpler EEPROM emulations. See [Section 5.3: EEPROM emulation timing](#) for further details.

4.8 Real-time considerations

The provided implementation of the EEPROM emulation firmware runs from internal flash memory, thus accesses to the same flash memory bank are stalled during flash memory erase or programming operations (EEPROM initialization, variable update, or page erase).

As a consequence, the application code is not executed and interrupts cannot be served during a maximum time equal to the longest page-erase operation duration for your STM32 product (see the applicable datasheet). This behavior may be acceptable for many applications, however for applications with real-time constraints, the user needs to take corrective action.

4.8.1 Device embedding flash memory with RWW (read while write) capability

With STM32 devices embedding a dual-bank flash memory, it is recommended to put the critical routines (vector table, critical interrupt service routines) in one bank and the area used for EEPROM emulation in the other bank. The flash memory area in the bank used for EEPROM emulation can still be used, but with execution delays during flash memory write and erase operations.

Note: To know whether your STM32 device supports flash memory with RWW capability, refer to the product datasheet.

4.8.2 Running the critical processes from the internal RAM

Another way to fulfill real-time constraints is to run the critical code from internal RAM:

1. Relocate the vector table into the internal RAM.
2. Execute all critical processes and interrupt service routines from the internal RAM. The compiler provides a keyword to locate the functions in RAM. The functions are copied from the flash memory to the RAM at system startup just like any initialized variable.

Note: It is important to note that for a RAM function, all used data, and called functions need to be located within the RAM.

4.8.3 Dual core considerations

On some STM32 devices, two cores are available (refer to the applicable STM32 product datasheet and reference manual for more information). This is the case for a majority of the products from the STM32 wireless series for example.

STM32WB series MCUs integrate a main core (CPU1, Arm Cortex-M4) alongside a second core (CPU2, Arm Cortex-M0+) that is dedicated to radio. Here we consider an example in a radio context, where CPU1 runs the EEPROM emulation and needs to process a flash memory operation.

To achieve this while avoiding conflicts with CPU2, CPU1 must check that the radio application running on CPU2:

- is not currently accessing flash memory.
- does not urgently need to access flash memory.

Note: Erase operations in particular take a relatively long time regarding the radio protocol specifications. Therefore, a mechanism must also be implemented to avoid the start of a long CPU1 operation (such as an erase) if CPU2 has already planned an urgent flash memory operation.

Application developers adapting the EEPROM emulation application for a dual-core product may need to implement these verifications.

For instance, for STM32WB series MCUs it is possible to use the following features:

- The hardware semaphore feature (HSEM)
- The CRPF bit from the PWR_EXTSCR register bit (set when a critical radio operation is ongoing)

Both are available on STM32WB series MCUs.

Note: The same problematic applies to some dual-core applications other than radio.

For STM32WB series products integrating a Bluetooth® LE stack, a 'flash access management' mechanism, based principally on the hardware semaphore feature, is described in document [6].

For the P-NUCLEO-WB55 Nucleo board, two examples are provided in the X-CUBE-EEPROM firmware package (see [Section 5.1.2: STM32Cube expansion software \(X-CUBE-EEPROM\)](#)):

- An example that simply exploits the EEPROM emulation driver (similar to the examples for other series).
- An example that maintains a Bluetooth® LE connection and communication while processing EEPROM operations. It follows the flash memory access timing rules in document [6], and can be used as a reference for implementing this mechanism.

To determine the availability of the above mechanisms for other STMicroelectronics MCUs with RF stacks, refer to their datasheets and other related documentation.

4.9 Cleaning up the flash memory in interrupt or polling mode

Once the valid elements are transferred to a new set of pages, the previous set of pages needs to be erased. The EEPROM emulation driver raises a flag requesting the main application to clean up the flash memory. This clean up can be delayed by the application program until the real time constraints on the application are lowered (for instance, before going to low power mode).

This flash memory cleanup can be done in polling mode (by calling the `EE_CleanUp` function) meaning that the cleanup function does not return before one set of flash memory pages used for EEPROM emulation is erased. It can also be done in interrupt mode (by calling the `EE_CleanUp_IT` function) meaning that the cleanup function returns immediately and following page erases are done by subsequent interrupt service routines.

4.10 Cache coherency maintenance

For most STM32 microcontroller series, the flash memory content is not intended to be modified while the application is running. Thus, on some STM32 microcontrollers, if flash memory data is cached and then modified, this modification is ignored by the cache, and its content becomes incoherent (the data in the cache is different to its new value in flash memory). The X-CUBE-EEPROM firmware implements code to keep the caches coherent so that the user can activate the caches as they wish (instruction cache, data cache, prefetch buffer and so on. Refer to your product reference manual to see which cache is proposed).

For example, for the STM32L5 series X-CUBE-EEPROM example, the MPU (memory protection unit) is used to define the EEPROM flash memory area as noncacheable. Another example is the STM32L4 series, where the DCACHE is flushed after page-erasing operations, as is advised by its reference manual.

5 API and application examples

5.1 EEPROM emulation software description

This section describes the driver implemented for EEPROM emulation using the STM32Cube firmware for your device series, provided by STMicroelectronics.

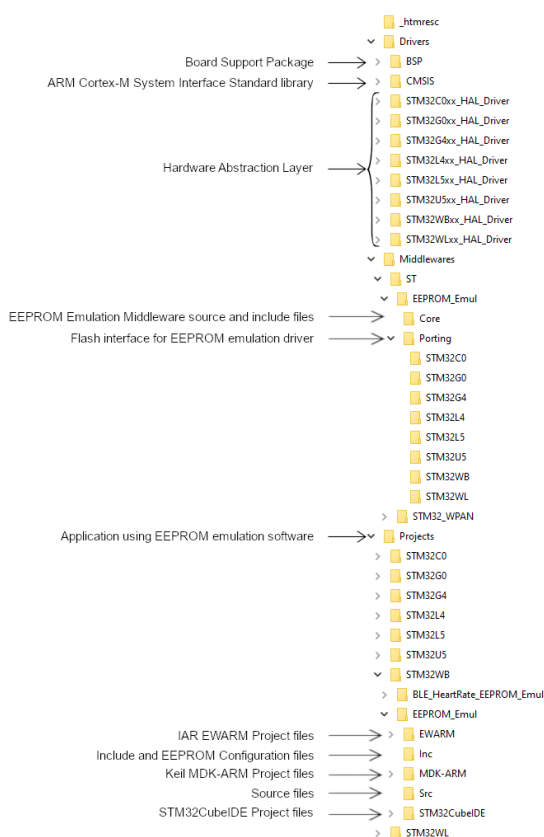
5.1.1 Key features

- User-configured EEPROM size.
- Supports 8-bit, 16-bit, 32-bit (and 96-bit variables on the STM32U5 and STM32H5 series).
- Increased EEPROM memory endurance versus flash memory endurance.
- Wear-leveling algorithm.
- Possible interrupt servicing during program and erase operations.
- Robust against asynchronous resets and power failures.
- Optional protection implementation for flash memory sharing between cores in multicore STM32 devices (for example STM32WB series products).

5.1.2 STM32Cube expansion software (X-CUBE-EEPROM)

The EEPROM emulation software is provided as a dedicated software package called X-CUBE-EEPROM. It is classified as a middleware utility to be compliant with all development boards and product series.

Figure 6. Directory tree



A sample demonstration project using the EEPROM emulation driver is also supplied to demonstrate how to manage 1000 nonvolatile variables. Sample projects are provided for the boards listed in Table 7. They are located in the Projects\STM32xx\EEPROM_Emul package folders, and can easily be tailored to other boards using the same STM32 MCU series.

One more example is provided for the P-NUCLEO-WB55.Nucleo board. It is located at `Projects\STM32WB\BLE_HeartRate_EEPROM_Emul\P-NUCLEO-WB55.Nucleo`. The Nucleo board maintains a Bluetooth® LE connection and communication in addition to processing EEPROM operations. By using the `DUALCORE_FLASH_SHARING` preprocessor-defines symbol, this example follows the flash memory sharing mechanism and timing rules described in [Section 4.8.3: Dual core considerations](#).

Table 7. Boards covered

STM32 series	Boards covered
STM32L4 series	STM32L476G-Discovery
STM32L4+ series	STM32L4R5ZI_Nucleo
STM32L5 series	NUCLEO-L552ZE-Q
STM32G0 series	NUCLEO-G071RB
STM32G4 series	NUCLEO-G431RB, NUCLEO-G474RE
STM32WB series	P-NUCLEO-WB55
STM32WL series	NUCLEO-WL33CC1, NUCLEO-WL55JC
STM32U0 series	NUCLEO-U031R8
STM32U3 series	NUCLEO-U385RG-Q
STM32U5 series	NUCLEO-U575ZI
STM32C0 series	NUCLEO-C031C6
STM32H5 series	NUCLEO-H563ZI

The project contains four source files:

- **`eeeprom_emul.c`**: contains the EEPROM emulation firmware functions that can be called from the user program:

```

EE_Format
EE_Init
EE_ReadVariable32bits
EE_WriteVariable32bits
EE_ReadVariable16bits
EE_WriteVariable16bits
EE_ReadVariable8bits
EE_WriteVariable8bits
EE_Status EE_ReadVariable64bits
EE_Status EE_WriteVariable64bits
EE_Status EE_ReadVariable96bits
EE_Status EE_WriteVariable96bits
EE_CleanUp
EE_CleanUp_IT
EE_DeleteCorruptedFlashAddress

```
- **`flash_interface.c`**: contains the functions needed to handle the STM32 flash specific features.
- **`main.c`**: this application program is an example using the described routines to configure, write to, and read from the emulated EEPROM.
- **`stm32xxx_it.c`**: shows an example of interrupt service file using the `EE_DeleteCorruptedFlashAddress` function in the NMI handler.

For the Bluetooth® LE STM32WB series example, `BLE_HeartRate_EEPROM_Emul`, the code illustrating the driver exploitation can be found in the `Core/Src/app_entry.c` file. In particular, the functions `EEPROM_Emul_Init` and `EEPROM_Emul_Operation` are examples of sequencer tasks for EEPROM driver initialization and exploitation (read and write operations).

5.1.3

User defines

The EEPROM emulation algorithm parameters need to be configured according to the application needs. They are located in the `eeeprom_emul_conf.h` file:

- **NB_OF_VARIABLES** (default 1000, 100 for STM32C0 series(a)): Number of nonvolatile elements, each element value being 8-, 16-, 32-or 96-bit.
- **START_PAGE_ADDRESS**: Address of the first flash memory page used for EEPROM emulation. For dual-bank capable devices, the first address of the second bank is a good example value for this parameter to benefit from this feature (see [Section 4.8: Real-time considerations](#)).
- **CYCLES_NUMBER** (default 1): The number of kcycles, X, for the equivalent EEPROM endurance. If CYCLES_NUMBER equals 10, the emulated EEPROM has an equivalent endurance of 10 x X kcycles. X is the flash memory endurance for the product used and listed in [Table 5. Flash memory endurance](#).
- **GUARD_PAGES_NUMBER** (default 2): Number of guard pages used to reduce pressure on flash memory. This number has to be even.
- **CRC_POLYNOMIAL_LENGTH** (default 16): No need to modify in most cases. A 16-bit CRC is optimized in terms of computational speed and flash memory size, and offers a very good detection rate.
- **CRC_POLYNOMIAL_VALUE** (default 0x8005): No need to modify in most cases. This ANSI CRC is optimized in terms of computational speed and flash memory size, and offers a very good detection rate.

To activate the dual-core flash memory sharing mechanism in the context of the Bluetooth® LE stack of the STM32WB series, DUALCORE_FLASH_SHARING must be defined as a preprocessor symbol in user software.

5.1.4

User API definition

The set of functions that can be called by the applicative program are described in [Table 8](#). They are defined in the *eeeprom_emul.c* module.

Table 8. API definition

Function name	Description
EE_Format	Erases all flash pages used for EEPROM emulation and writes an ACTIVE header to the first page.
EE_Init	Configures the EEPROM emulation variables to their initial state and restores the flash pages to a known good state in case of asynchronous reset or power loss during flash memory write or erase operation. Erases the flash pages that need to be erased (for instance pages not fully erased in ERASE state, pages in ERASING state). The EE_FORCE_ERASE parameter has to be used by default. If the application guarantees that no asynchronous reset nor power loss can happen during flash write or erase operations, then the EE_CONDITIONAL_ERASE parameter can be used. Refer to Section 4.7.2: Page header recovery. After reset, this function needs to be systematically called prior to accessing the emulated EEPROM.
EE_ReadVariableXXbits	This function updates the data value corresponding to the virtual address passed as a parameter. Only the last stored element is read. It returns a status equal to EE_OK unless it cannot find any data at the given virtual address. Three functions are implemented to deal with all supported data sizes (XX = 8, 16, 32 or 64).
EE_WriteVariableXXbits	This function updates the EEPROM at a given virtual address with the data value passed as parameters. If the set of pages used for EEPROM emulation is full, it triggers a flash memory page transfer and returns a Status equal to EE_CLEANUP_REQUIRED. In the case of the Bluetooth® LE application for the NUCLEO-WB55 Nucleo board (DUALCORE_FLASH_SHARING is defined), it can return EE_FLASH_USED when the flash memory is used by CPU2⁽¹⁾ Three functions are implemented to deal with all supported data sizes (XX = 8, 16, 32 or 64).

Function name	Description
EE_CleanUp	Erases the set of pages in ERASING state. The application program can call this function when real-time constraints are low. The flash cleanup is done in polling mode so this function does not return before one set of flash memory pages used for EEPROM emulation is erased.
EE_CleanUp_IT	Erases the set of pages in ERASING state. The flash cleanup is started in interrupt mode so this function returns immediately and following page erase is done by subsequent interrupt service routines.
FI_DeleteCorruptedFlashAddress	This function can be called under NMI to delete a corrupted flash address found by the flash memory ECCD in order to clear this flash interface fault Section 4.7.1: Data recovery .

1. *EE_WriteVariableXX bits can return EE_FLASH_USED only in the case of direct EEPROM variable writing and not in the case of a page transfer, a page header update, or the initialization process. Also, when this value is returned, the driver can activate the HSEM interrupt so that an interrupt is raised when the CPU2 releases the semaphore associated to the flash memory device being used. The user program can then do something else while waiting for the semaphore release.*

5.2 EEPROM emulation memory footprint

[Table 9](#) details the footprint of the EEPROM emulation driver in terms of flash memory, RAM, and stack size for some of the products covered by this application note.

The figures shown have been determined using the IAR EWARM 8.42.2 (or later) tool with a high size optimization level. Note that this algorithm does not use heap.

Table 9. Memory footprint for EEPROM emulation mechanism

STM32 series or line	Required ⁽¹⁾ code and data size in bytes	
	Flash	SRAM
STM32L4 series	4256	12
STM32L4+ series	4286	12
STM32L5 series	4728	12
STM32G0 series	3812	12
STM32G4 series	4946	12
STM32WB series	5392	16
STM32WB series ⁽²⁾	3328	14
STM32WL3x line	3396	8
STM32U0 series	4920	12
STM32U3 series	4672	12
STM32U5 series	5548	12
STM32C0 series	5098	8
STM32H5 series	5300	12
STM32H5 series using high-cycle data area	2488	12

1. Based on 1000 elements (16-bit address, 16-bit CRC, and 32-bit data).

2. Example with Bluetooth® LE communication.

[Table 9](#) does not include the flash memory size used for EEPROM emulation itself. Refer to the formula given in [Section 4.6: Computing the required size of flash memory for EEPROM emulation](#) to calculate the corresponding flash memory size.

Note also that the application program (*main.c*) implements the `VarDataTab[]` variable to store the EEPROM element values. Its size is 4000 bytes for 1000 32-bit elements.

For flexibility, this variable is placed in RAM in the provided example. Moreover, `VarDataTab[]` can easily be removed as most user applications do not need to have a RAM copy of the elements stored in flash memory.

5.3 EEPROM emulation timing

This section describes the timing parameters associated with the EEPROM emulation driver based on 1000 32-bit variables, except for the STM32U5 and STM32H5 series, which have 1000 96-bit variables. Moreover, the STM32H5 series including high-cycle data area have 1000 32-bit variables.

All timing measurements are performed with the following conditions:

- At $V_{DD} = 3.3\text{ V}$
- With execution from flash memory (from the bank not used for EEPROM variables, when dual bank is used)
- At room temperature
- Based on 100 operations for each type of operation.

Table 10 lists the timing values for the EEPROM emulation driver in these conditions for some of the products covered by this application note. In other conditions, for instance for a different number of variables, cycles endurance or a different MCU frequency, these timings can differ significantly.

Table 10. EEPROM emulation timings for different targets

Target	Conditions	Operation	Min.	Mean	Max.
STM32L476G-DISCO	• System clock = 80 MHz • ART enabled • flash memory prefetch disabled • Endurance = 100 kcycles	EEPROM initialization with forced erase ⁽¹⁾	904.4 ms	1.2215 s	2.0212 s
		EEPROM initialization with conditional erase ⁽¹⁾⁽²⁾	6.1 ms	365.56 ms	1.2499 s
		Write operation in EEPROM	90 μ s	- ⁽³⁾	344.90 ms
		Read operation from EEPROM ⁽⁴⁾	9.3 μ s	47.412 μ s	311.5 μ s
		EEPROM cleanup ⁽¹⁾	900.13 ms	900.27 ms	900.38 ms
		EEPROM cleanup IT ⁽¹⁾	900 ms	900.19 ms	900.38 ms
NUCLEO-L4R5ZI	• System clock = 80 MHz⁽⁵⁾ • ART enabled • flash memory prefetch disabled • Endurance = 100 kcycles	EEPROM initialization with forced erase ⁽¹⁾	509.6 ms	741.69 ms	884 ms
		EEPROM initialization with conditional erase ⁽¹⁾⁽²⁾	5.8 ms	466.91 ms	776.9 ms
		Write operation in EEPROM	90 μ s	- ⁽³⁾	307.20 ms
		Read operation from EEPROM ⁽⁴⁾	9 μ s	75.639 μ s	314.3 μ s
		EEPROM cleanup ⁽¹⁾	461.8 ms	461.90 ms	462 ms
		EEPROM cleanup IT ⁽¹⁾	461.8 ms	461.86 ms	462 ms
NUCLEO-L552ZE-Q	• System clock = 110 MHz • ICACHE disabled • Endurance = 10 kcycles	EEPROM initialization with forced erase ⁽¹⁾	111.1 ms	365.10 ms	670 ms
		EEPROM initialization with conditional erase ⁽¹⁾⁽²⁾	1.5 ms	267.5 ms	680 ms
		Write operation in EEPROM	90 μ s	- ⁽³⁾	580.18 ms
		Read operation from EEPROM ⁽⁴⁾	10 μ s	149.3 μ s	485.6 μ s
		EEPROM cleanup ⁽¹⁾	109.80 ms	109.85 ms	109.89 ms
		EEPROM cleanup IT ⁽¹⁾	109.81 ms	109.85 ms	109.88 ms
NUCLEO-G071RB	• System clock = 64 MHz • Endurance = 10 kcycles	EEPROM initialization with forced erase ⁽¹⁾	111.4 ms	251.17 ms	814.4 ms
		EEPROM initialization with conditional erase ⁽¹⁾⁽²⁾	1.47 ms	233.43 ms	792.9 ms
		Write operation in EEPROM	90 μ s	- ⁽³⁾	702.18 ms

Target	Conditions	Operation	Min.	Mean	Max.
NUCLEO-G071RB	- System clock = 64 MHz - Endurance = 10 kcycles	Read operation from EEPROM ⁽⁴⁾	14 µs	102.16 µs	592 µs
		EEPROM cleanup ⁽¹⁾	110.37 ms	110.39 ms	110.40 ms
		EEPROM cleanup IT ⁽¹⁾	110.40 ms	110.41 ms	110.43 ms
NUCLEO-G431RB	- System clock = 170 MHz - Endurance = 10 kcycles	EEPROM initialization with forced erase ⁽¹⁾	110.3 ms	179.33 ms	349.4 ms
		EEPROM initialization with conditional erase ⁽¹⁾⁽²⁾	600 µs	160.90 ms	356.3 ms
		Write operation in EEPROM	90 µs	- ⁽³⁾	260.67 ms
		Read operation from EEPROM ⁽⁴⁾	6 µs	74.242 µs	170 µs
		EEPROM cleanup ⁽¹⁾	109.86 ms	109.89 ms	109.89 ms
		EEPROM cleanup IT ⁽¹⁾	109.89 ms	109.90 ms	109.92 ms
P-NUCLEO-WB55.Nucleo ⁽⁶⁾	- System clock = 64 MHz - Endurance = 10 kcycles	EEPROM initialization with forced erase ⁽¹⁾	67.27 ms	276.83 ms	561.32 ms
		EEPROM initialization with conditional erase ⁽¹⁾⁽²⁾	1.86 ms	258.65 ms	539.78 ms
		Write operation in EEPROM	90 µs	- ⁽³⁾	476.26 ms
		Read operation from EEPROM ⁽⁴⁾	11 µs	289.25 µs	570 µs
		EEPROM cleanup ⁽¹⁾	65.82 ms	65.867 ms	65.92 ms
		EEPROM cleanup IT ⁽¹⁾	64.90 ms	64.966 ms	65.92 ms
NUCLEO-WL33CC1 ⁽⁷⁾	- System clock = 64 MHz - Endurance = 10 kcycles	EEPROM initialization with forced erase ⁽¹⁾	108.1 ms	436 ms	1.007 s
		Write operation in EEPROM	141.2 us	- ⁽³⁾	60.17 ms
		Read operation from EEPROM ⁽⁴⁾	23.2 us	92.36 us	733.5 us
		EEPROM cleanup ⁽¹⁾	42.137 ms	42.139 ms	42.142 ms
		EEPROM cleanup IT ⁽¹⁾	42.137 ms	42.140 ms	42.142 ms
NUCLEO-U031R8	- System clock = 48 MHz - Endurance = 10 kcycles	EEPROM initialization with forced erase ⁽¹⁾	112.4 ms	426.97 ms	922.4 ms
		EEPROM initialization with conditional erase ⁽¹⁾⁽²⁾	2.26 ms	385.4 ms	920.5 ms
		Write operation in EEPROM	115.4 µs	- ⁽³⁾	709.1 ms
		Read operation from EEPROM ⁽⁴⁾	26.89 µs	92.18 µs	157.5 µs
		EEPROM cleanup ⁽¹⁾	109.8 ms	109.87 ms	110 ms
		EEPROM cleanup IT ⁽¹⁾	109.9 ms	109.92 ms	110 ms
NUCLEO-U385RG-Q	- System clock = 96 MHz - Endurance = 10 kcycles	EEPROM initialization with forced erase ⁽¹⁾	36.02 ms	129.92 ms	347 ms
		EEPROM initialization with conditional erase ⁽¹⁾⁽²⁾	0.500 ms	129.44 ms	368 ms
		Write operation in EEPROM	77.74 µs	- ⁽³⁾	277.5 ms
		Read operation from EEPROM ⁽⁴⁾	5.1 µs	40.56 µs	94.4 µs
		EEPROM cleanup ⁽¹⁾	35.92 ms	35.97 ms	35.98 ms

Target	Conditions	Operation	Min.	Mean	Max.
NUCLEO-U385RG-Q	- System clock = 96 MHz - Endurance = 10 kcycles	EEPROM cleanup IT ⁽¹⁾	35.92 ms	35.97 ms	36.02 ms
NUCLEO-U575ZI-Q	- System clock = 160 MHz - ICACHE disabled - Endurance = 10 kcycles	EEPROM initialization with forced erase ⁽¹⁾	4 ms	207 ms	410 ms
		EEPROM initialization with conditional erase ⁽¹⁾⁽²⁾	1.6 ms	164.8 ms	328 ms
		Write operation in EEPROM	129 µs	- ⁽³⁾	540 ms
		Read operation from EEPROM ⁽⁴⁾	8 µs	234 µs	460 µs
		EEPROM cleanup ⁽¹⁾	4.623 ms	4.74 ms	4.852 ms
		EEPROM cleanup IT ⁽¹⁾	4.623 ms	4.73 ms	4.844 ms
NUCLEO-C031C6	- System clock = 48 MHz - Endurance = 10 kcycles	EEPROM initialization with forced erase ⁽¹⁾	88.8 ms	557 ms	926.9 ms
		EEPROM initialization with conditional erase ⁽¹⁾⁽²⁾	23.3 ms	519 ms	938.9 ms
		Write operation in EEPROM	116.7 µs	- ⁽³⁾	838.5 ms
		Read operation from EEPROM ⁽⁴⁾	28.4 µs	92.65 µs	156.9 µs
		EEPROM cleanup ⁽¹⁾	87.8 ms	87.83 ms	87.89 ms
		EEPROM cleanup IT ⁽¹⁾	87.81 ms	87.83 ms	87.86 ms
NUCLEO-H563ZI	- System clock = 240 MHz - Endurance = 10 kcycles	EEPROM initialization with forced erase ⁽¹⁾	5.126 ms	61.25 ms	173.1 ms
		EEPROM initialization with conditional erase ⁽¹⁾⁽²⁾	0.415 ms	56.48 ms	167.7 ms
		Write operation in EEPROM	33.72 µs	- ⁽³⁾	174.1 ms
		Read operation from EEPROM ⁽⁴⁾	3.18 µs	29.7 µs	143.1 µs
		EEPROM cleanup ⁽¹⁾	4.931 ms	4.94 ms	4.939 ms
		EEPROM cleanup IT ⁽¹⁾	4.931 ms	4.93 ms	4.938 ms
NUCLEO-H563ZI using high-cycle data area	- System clock = 240 MHz - Endurance = 100 kcycles	EEPROM initialization with forced erase ⁽¹⁾	175.3 ms	333.9 ms	519.5 ms
		EEPROM initialization with conditional erase ⁽¹⁾⁽²⁾	160.5 ms	304.4 ms	457.1 ms
		Write operation in EEPROM	126.2 µs	- ⁽³⁾	370.2 ms
		Read operation from EEPROM ⁽⁴⁾	3.802 µs	114.63 µs	226.1 µs
		EEPROM cleanup ⁽¹⁾	4.922 ms	4.93 ms	4.939 ms
		EEPROM cleanup IT ⁽¹⁾	4.922 ms	4.93 ms	4.943 ms

1. The maximum initialization time and cleanup time depend on the number of flash memory EEPROM pages used, and the time to erase one page. (See the section "Flash memory characteristics" in the STM32 product datasheet). The time taken to erase flash memory EEPROM pages = number of pages * tERASE.
2. When the application ensures the EEPROM emulation cannot be interrupted by an asynchronous reset or power failure, a conditional erase can be used. It is more efficient in terms of execution time and also in terms of EEPROM endurance (refer to [Section 4.7.2: Page header recovery](#) for more details).
3. The typical write time is close to the minimal write time as there is no data transfer in most cases. The maximum value refers to a write operation that generates a data transfer.
4. The minimum value refers to a read operation of the last element stored in the flash memory and the maximum value refers to a read operation of the first element stored in the flash memory.

5. *Timing measurements for the STM32L4+ series board (NUCLEO-L4R5ZI) has been done for an 80 MHz system clock so that a comparison can be done with the STM32L4 series (NUCLEO-L476RG). However, the NUCLEO-L4R5ZI can be pushed up to 120 MHz while the NUCLEO-L476RG is limited at 80 MHz. The measurements for the other boards are done for the maximum system clock.*
6. *Data taken from the example without Bluetooth® LE capabilities.*
7. *For NUCLEO-WL33C1, EEPROM initialization timing with conditional erase is not specified, because the device does not support differentiation between a wake-up from Deepstop mode and an asynchronous reset or power failure.*

Note: These timing measurements are provided to give the reader an idea of the emulation firmware performance for different STM32 series. They are based on 100 operations for each type of operation. Thus, they are not an absolute reference. It is possible to find different minimums, maximums and even means (especially if the measurement conditions are modified).

6 Embedded application aspects

This section provides advice on how to overcome software limitations in embedded applications and how to fulfill the needs of different applications.

6.1 Data retention

The flash memory data retention is typically 30 years at 85°C after cycling 1000 times (refer to the STM32 product datasheet for more information). Nothing specific is done in the EEPROM emulation driver to increase the EEPROM data retention. However, when a page transfer is initiated, all data (including long-lived data) is copied to a new flash memory page. This ensures an EEPROM data retention significantly higher than the original flash memory data retention.

6.2 Detecting flash memory operations interrupted by a reset on STM32U5 devices

The STM32U5 series FLASH_OPSR register indicates whether a flash memory operation was interrupted by a reset or not.

Note: The current X-CUBE-EEPROM release does not use this register.

In association with power-failure detection (see [Section 6.3](#)), this allows the `EE_Init` function to be called with the `EE_CONDITIONAL_ERASE` parameter, avoiding the need to use the `EE_FORCE_ERASE` parameter.

Additionally, the current EEPROM emulation erases the flash memory first empty line after each reset. This operation is avoided by using of the FLASH_OPSR register.

6.3 Detecting power failures

If no asynchronous reset can be generated by the application, the only way to corrupt the flash memory programming, or erasing operations is the case of power failure. The application can use the PVD not to generate a flash memory write or erase operation during the VDD ramp down. An example of this PVD (programmable voltage detector) usage is provided in the EEPROM example main application.

The decoupling capacitors or battery must be sized to provide enough power to the chip during the full write or erase operation and before the supply voltage goes below 1.7 V (minimal operating voltage) or before a BOR (brownout reset) is generated. In this case, the `EE_CONDITIONAL_ERASE` parameter of the `EE_Init` routine can be used. In all other cases, the `EE_FORCE_ERASE` parameter has to be used.

Note: PVD is not available on STM32C0 series.

6.4 Reducing worst case access times

An EEPROM read command consists of performing flash memory reads from the highest to the lowest address in the ACTIVE and VALID pages. If the data to retrieve was the first to be written, this process can be quite long. Similarly, the page transfer searches for all virtual addresses used during EEPROM emulation and looks for corresponding data values. These read and transfer times can be quite long when the size of emulated EEPROM is large.

The worst-case read and write access times can be reduced by implementing a LUT (look up table) storing the element values in RAM. Although implementing a LUT does not affect the typical access times and is quite costly in terms of RAM space, it can still be an option implemented in a future revision of the EEPROM emulation driver.

7 Conclusion

The STM32 devices internal flash memory can be used advantageously in many applications to emulate an EEPROM. Even though the memory characteristics are quite different, this application note has shown that emulated EEPROM competes with real EEPROM in terms of:

- Memory size
- Read and write access times
- Driver usage simplicity and flexibility
- Memory endurance
- Data retention
- Robustness to asynchronous resets and power failures
- Reliability
- Cost by removing the need for an external component

Revision history

Table 11. Document revision history

Date	Version	Changes
11-Jul-2017	1	Initial release.
20-Sep-2018	2	Updated document to scope STM32L4 and STM32L4+ series devices. Updated: - Figure 1: Page status evolution - Figure 2: Page status valid transitions - Section 3.3: Page and variable format - Figure 3: Flash page and EEPROM variable format (except STM32U5 series) - Section 4.4: Cycling capability: EEPROM endurance improvement - Table 7: Flash usage for a 4000-byte emulated EEPROM (STM32L4/L4+) - Section : As a consequence, the application code is not executed and interrupts cannot be served during a maximum time equal to the longest page-erase operation duration for your STM32 product (see the applicable datasheet). This behavior may be acceptable for many applications, however for applications with real-time constraints, the user needs to take corrective action. - Section 5.1.3: User defines - Section 5.2: EEPROM emulation memory footprint - Section 5.3: EEPROM emulation timing Added Section 4.6: Computing the required size of flash memory for EEPROM emulation.
01-Jun-2020	3	Updated: - Document title - Introduction - Section 3.3: Page and variable format and Table 4: Flash memory properties - Section 3.4: Simple use case - Section 4.4: Cycling capability: EEPROM endurance improvement and Table 6: Flash memory endurance - Section 4.6: Computing the required size of flash memory for EEPROM emulation - Section 4.7.1: Data recovery - Section 4.8.3: Dual core considerations - Section 5.1.1: Key features - Section 5.1.2: STM32Cube expansion software (X-CUBEEEPROM), Figure 6: Directory tree, and Table 8: Boards covered - Section 5.1.3: User defines - Section 5.1.4: User API definition and Table 9: API definition - Section 5.2: EEPROM emulation memory footprint and Table 10: Memory footprint for EEPROM emulation mechanism - Section 5.3: EEPROM emulation timing and Table 11: EEPROM emulation timings for different targets.
03-Nov-2020	4	Extended document scope to STM32WL series. Updated: - Introduction - Table 1: Applicable products - Table 4: Flash memory properties - Table 6: Flash memory endurance - Table 8: Boards covered Added Section 4.10: Cache coherency maintenance.
16-Dec-2021	5	Extended document scope to STM32U5 series. Updated:

Date	Version	Changes
		• Introduction • Section 3.1: Principle • Section 3.3: Page and variable format • Section 3.3: Page and variable format • Section 4.1: Data granularity management • Section 4.7.1: Data recovery • Section 5.1.1: Key features • Section 5.1.2: STM32Cube expansion software (X-CUBEEEPROM) • Table 6: Directory tree • Section 5.1.3: User defines • Section 5.3: EEPROM emulation timing • Table 9: API definition • Table 10: Memory footprint for EEPROM emulation mechanism • Table 11: EEPROM emulation timings for different targets Added Section : The flash memory data retention is typically 30 years at 85°C after cycling 1000 times (refer to the STM32 product datasheet for more information). Nothing specific is done in the EEPROM emulation driver to increase the EEPROM data retention. However, when a page transfer is initiated, all data (including longlived data) is copied to a new flash memory page. This ensures an EEPROM data retention significantly higher than the original flash memory data retention.. Differentiated Flash memory page sizes between STM32WB55xx and STM32WB15xx series in Table 4: Flash memory properties.
12-Jan-2023	6	Updated: • Table 1: Applicable products • Table 4: Flash memory properties • Table 6: Flash memory endurance • Table 8: Boards covered • Table 10: Memory footprint for EEPROM emulation mechanism • Table 11: EEPROM emulation timings for different targets • Figure 6: Directory tree • Section 6.3: Detecting power failures Added Section : The flash memory data retention is typically 30 years at 85°C after cycling 1000 times (refer to the STM32 product datasheet for more information). Nothing specific is done in the EEPROM emulation driver to increase the EEPROM data retention. However, when a page transfer is initiated, all data (including longlived data) is copied to a new flash memory page. This ensures an EEPROM data retention significantly higher than the original flash memory data retention.
18-Jul-2023	7	Updated: • Document title • Section : Introduction • Table 1: Applicable products • Table 4: Flash memory properties • Section 4.1: Data granularity management • Table 6: Flash memory endurance • Section 5.1.1: Key features • Table 8: Boards covered • Table 10: Memory footprint for EEPROM emulation mechanism • Table 5.3: EEPROM emulation timing • Table 11: EEPROM emulation timings for different targets Added Section 4.5: High-cycle data area of flash memory
25-Sep-2023	8	Updated Table 11: EEPROM emulation timings for different targets
15-Mar-2024	9	Extended document scope to STM32U0 series.
13-Mar-2025	10	Extended document scope to STM32U3 series.
18-Mar-2025	11	Updated Table 3. Flash memory properties.

Date	Version	Changes
06-Jul-2026	12	Extended document scope to NUCLEO-WL33CC1 and updated Section 1: General information .

Contents

1	General information	2
2	Main differences between external and emulated EEPROM	3
2.1	Difference in write access time	3
2.2	Programming and erase operations	3
3	Implementing EEPROM emulation	5
3.1	Principle	5
3.2	Page status valid transitions	5
3.3	Page and variable format	6
3.4	Simple use case	9
3.5	Reading data	9
4	Advanced features	10
4.1	Data granularity management	10
4.2	Wear leveling algorithm and flash memory page allocation	10
4.3	Guard pages	10
4.4	Cycling capability: EEPROM endurance improvement	10
4.5	High-cycle data area of flash memory	12
4.6	Computing the required size of flash memory for EEPROM emulation	12
4.7	EEPROM emulation robustness	13
4.7.1	Data recovery	13
4.7.2	Page header recovery	14
4.8	Real-time considerations	14
4.8.1	Device embedding flash memory with RWW (read while write) capability	14
4.8.2	Running the critical processes from the internal RAM	14
4.8.3	Dual core considerations	14
4.9	Cleaning up the flash memory in interrupt or polling mode	15
4.10	Cache coherency maintenance	15
5	API and application examples	16
5.1	EEPROM emulation software description	16
5.1.1	Key features	16
5.1.2	STM32Cube expansion software (X-CUBE-EEPROM)	16
5.1.3	User defines	17
5.1.4	User API definition	18
5.2	EEPROM emulation memory footprint	19
5.3	EEPROM emulation timing	20

6	Embedded application aspects	.24
6.1	Data retention	.24
6.2	Detecting flash memory operations interrupted by a reset on STM32U5 devices.	.24
6.3	Detecting power failures.	.24
6.4	Reducing worst case access times.	.24
7	Conclusion	.25
	Revision history	.26
	List of tables	.31
	List of figures.	.32

List of tables

Table 1.	Application products.	1
Table 2.	Differences between external and emulated EEPROM	3
Table 3.	Flash memory properties	6
Table 4.	Emulated EEPROM virtual addresses.	9
Table 5.	Flash memory endurance	11
Table 6.	Flash usage for a 4000-byte emulated EEPROM (STM32L4/L4+)	11
Table 7.	Boards covered	17
Table 8.	API definition.	18
Table 9.	Memory footprint for EEPROM emulation mechanism.	19
Table 10.	EEPROM emulation timings for different targets.	20
Table 11.	Document revision history	26

List of figures

Figure 1.	Page status evolution	5
Figure 2.	Page status valid transitions	6
Figure 3.	Flash page and EEPROM variable format (except STM32U5 series)	8
Figure 4.	STM32U5 series Flash page and EEPROM variable format	8
Figure 5.	Data update flow.	9
Figure 6.	Directory tree	16

IMPORTANT NOTICE – READ CAREFULLY

STMicroelectronics NV and its subsidiaries ("ST") reserve the right to make changes, corrections, enhancements, modifications, and improvements to ST products and/or to this document at any time without notice.

In the event of any conflict between the provisions of this document and the provisions of any contractual arrangement in force between the purchasers and ST, the provisions of such contractual arrangement shall prevail.

The purchasers should obtain the latest relevant information on ST products before placing orders. ST products are sold pursuant to ST's terms and conditions of sale in place at the time of order acknowledgment.

The purchasers are solely responsible for the choice, selection, and use of ST products and ST assumes no liability for application assistance or the design of the purchasers' products.

No license, express or implied, to any intellectual property right is granted by ST herein.

Resale of ST products with provisions different from the information set forth herein shall void any warranty granted by ST for such product.

If the purchasers identify an ST product that meets their functional and performance requirements but that is not designated for the purchasers' market segment, the purchasers shall contact ST for more information.

ST and the ST logo are trademarks of ST. For additional information about ST trademarks, refer to www.st.com/trademarks. All other product or service names are the property of their respective owners.

Information in this document supersedes and replaces information previously supplied in any prior versions of this document.

© 2026 STMicroelectronics – All rights reserved